

Trust by Construction: A Cross-Sector Reference Architecture for Governed Agentic AI

R. Srivastava

Submission: Expanded research draft for UNU Macau AI Conference 2026 — *AI × Education: AI for Learning, Learning for AI*, 25–26 November 2026, Macau SAR, China.

Proposed panel: Agentic AI in the Loop — From Autonomous Tools to Shared Capacity.

Keywords: agentic AI; reference architecture; secure data governance; privacy by construction; institutional sovereignty; fail-secure systems; system literacy; digital public goods.

Reference implementation: Apache-2.0, <https://github.com/genaiworks/fssai-ra>, version 1.0.0 reference baseline, audited working tree based on commit 5c587c361cf114125e39344caa6802efd4eaff11, with preserved author changes, the joined-path revision, and a key-secrecy and composition hardening pass (Section 9.2). Quantitative fixture results regenerate with `make results`; the new deployment requirements below are specifications, not measured results. This expanded draft is not a claim of conference acceptance or compliance with a final page limit.

ABSTRACT

AI governance still centres on models, yet harm arises in systems: an agent reads a record it should not see, sends a summary it should not release, or turns a plausible proposal into an irreversible action. This paper presents Trust by Construction, a sector-neutral reference architecture and development method for agentic AI, built on one falsifiable thesis: **intelligence is untrusted; power and data are mediated**. Two constitutional rules follow. A model may propose an action, but it cannot manufacture the authority to execute it. A model may request information, but it cannot manufacture the entitlement to see it, or launder what it saw.

The architecture places enforcement outside the model through a seven-field control contract, seven logical planes, and two mediators for action and disclosure. Its target privacy profile combines scoped tokens, per-subject encryption, approved endpoints, and independently authorized release. These guarantees depend on protected enforcement services and complete mediation; tokenization does not establish anonymity, and a signed model manifest does not prove the serving runtime is honest. A seven-stage lifecycle shows teams in any sector how to frame, contract, bind, falsify, promote, and operate a new AI capability. In synthetic fixtures, six action-domain packs contained 180 of 180 hostile scenarios and completed 58 of 58 benign tasks; four disclosure packs contained 106 of 106 hostile data flows and completed 15 of 15 legitimate flows. Separately, 104,997 bounded attempts found no counterexample within their declared search spaces. In an education demonstration with a deliberately malicious model, 25 of 25 falsifiers held and 300 red-team attempts produced 0 successes, while removing one control, or one declared pair, restored harm in 25 of 29 ablations. These are finite software observations, not security probabilities or evidence of institutional effectiveness. A further adversarial audit found missing assignment and instructor-confirmation checks despite a passing baseline suite. A new durable SQLite workflow repairs that joined path and tests static, adaptive-search and trained tabular-Q attacks, with authenticated approval, release and appeal. It remains a synthetic teaching implementation; hostile-code isolation, live-model evaluation and institutional effectiveness remain unverified.

1. Introduction

A company releases a confidential dataset; a hospital shares a record; a university corrects a transcript; a public body decides support. Different laws apply, yet each institution must answer the same questions: **what was allowed, who or what had authority, which evidence was used, what actually happened, and how can the outcome be challenged or reversed?** A safe model cannot answer those questions on behalf of an unsafe system.

Model-centred governance faces four structural problems that grow with capability. First, a model's intentions cannot be verified, and more capable models are more persuasive and harder to evaluate. Second, agents compose: an assistant delegates to a tool that calls another agent, and local correctness at every hop does not make a chain correct. Third, most harm in a corporate copilot or clinical assistant changes no state at all; it reads, summarises, and sends. Fourth, human review is a finite resource, and an approval given faster than a reviewer can think is a signature, not oversight.

Trust by Construction answers all four with one thesis and three commitments. *Untrusted intelligence*: treat every model as capable, persuasive, and possibly misaligned or manipulated. *Mediated power*: every effect and every flow of protected data passes a mediator whose authorization policy and credentials are outside model control. The model necessarily influences requests; it must not control the rules that decide them. *Evidenced trust*: every mediation is contracted, tested, and evidenced where it is relied on. Boundary checks offer a more explicit verification target than intentions, although the number of integrations and the trusted infrastructure can still make assurance expensive. The thesis is falsifiable within its stated assumptions: one admitted path on which model output supplies its own authority or entitlement is a counterexample. A compromised trusted mediator invalidates an assumption and requires a separate incident response; it must not be counted as an attack the design contains.

The paper makes five contributions:

1. **A reference architecture** that separates seven planes around an executable control contract and two mediators (Section 3).
2. **A governed request path in ten steps**, including a privacy pipeline of tokenization, envelope encryption, sensitivity-aware routing, and model attestation (Section 4).
3. **A transfer method**: domain packs that carry institutional meaning over one shared kernel, illustrated across five sectors (Section 5), and a seven-stage lifecycle for building any future AI capability (Section 6).
4. **Evidence that tests the architecture instead of restating it**: comparisons with conventional controls, ablation of every control, bounded model checking, stateful testing, and falsifiers, all regenerable offline (Section 7).
5. **A teaching method for system literacy**, so that policymakers, engineers, and learners can inspect the same boundaries (Section 8).

2. Background and related work

The architecture deliberately inherits rather than invents its mechanisms. The reference monitor and the principles of least privilege, complete mediation, and fail-safe defaults [12] place every access behind a mediator. Lattice information flow [13] lets restriction accumulate as data moves. Well-formed transactions and separation of duties [14] make integrity a property of the process, not of the actor. Zero trust architecture [1] removes implicit trust from network location.

Recent work applies these ideas to language-model agents. CaMeL separates control flow from untrusted data and attaches capabilities to values [15]; FIDES tracks confidentiality and integrity labels through an agent's planner [16]; AI control designs protocols that remain safe when the model actively tries to subvert them [17]. Governance frameworks supply the vocabulary of risk and management: the NIST AI RMF generative AI profile [2], ISO/IEC 42001 [6], the OWASP agentic top ten [3] and agent control standard [4], and MITRE ATLAS [5]. Data-protection law adds purpose limitation, pseudonymisation, and the right to erasure [18], and the EU AI Act adds obligations for high-risk systems [19]. Recent agent defences combine reference-monitor enforcement or information-flow control with selective human confirmation — symbolic privilege rules on every tool call [24] and dependency-screened tool execution [25] — so the contribution here is the cross-sector integration and transfer method, not the enforcement idea. Robustness claims for such defences do not survive without adaptive attackers, which break defences that reported near-perfect containment against static suites [21, 22], and three limiting results bound any such architecture: access control confines only explicit flows, leaving covert channels and inference from rare attributes [27, 28, 29]; differential privacy, not tokenization, is what would bound re-identification [30]; and a hard authorization boundary must not depend on a model honestly explaining itself, because chain-of-thought monitoring is optional and defeatable [32, 33].

Our contribution is an integration and transfer method: linking reference-monitor enforcement and information-flow controls to institutional purpose, review capacity, recovery, contestability, and executable evidence. We do not claim to invent mediation or to establish that prior work cannot support multiple agents. The comparison arms in Section 7

are local implementations of specified weaker policies, not evaluations of CaMeL, FIDES, or commercial products. Standards alignment supplies design requirements; it does not establish certification or legal compliance.

3. The architecture

3.1 Two constitutional rules

A model may propose an action; it cannot manufacture the authority to execute it. A model may request information; it cannot manufacture the entitlement to see it, or launder what it saw. The target deployment gives write credentials to the executor and record access and key-use permissions to the context gate and its custody service. Each use must recheck the applicable authorization state. Component-level fixtures exercise parts of this contract; the default HTTP profile enforces disclosure checks, while an opt-in in-memory privacy profile now additionally binds encrypted records, scoped tokens, strict model registration, and entitled restoration. This is reference integration evidence, not qualification of persistent custody or a production deployment. Authority only narrows as agents delegate; restriction only accumulates as data flows. Trustworthiness becomes an institutional capability that can be specified and tested, not a property bought with a model, a region, or a certificate.

3.2 The control contract

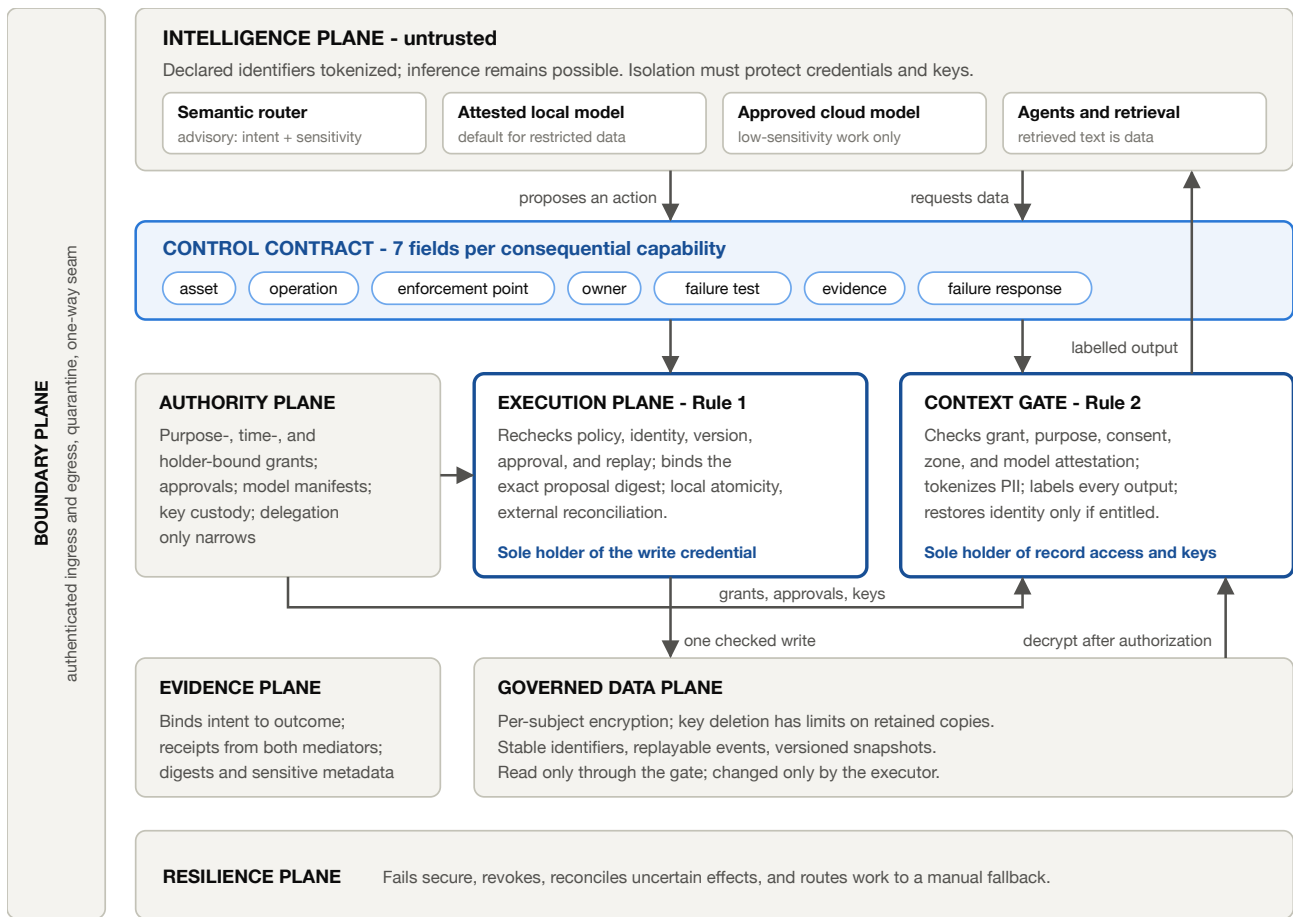
The narrow waist of the architecture is an executable contract between policy and implementation. For each consequential capability, its **seven fields** name the protected asset, permitted operation, independent enforcement point, accountable owner, failure test, evidence artifact, and failure response (Table 1). If an institution cannot fill every field, it has found an unresolved governance decision rather than a capability ready for automation. The contract is machine-readable, and each requirement is bound to a check that is itself checked to exist.

Table 1. The seven fields of the control contract, with one capability from the healthcare domain pack.

Field	Question it answers	Example: authorize treatment access
Protected asset	What could be harmed?	A patient's treatment record
Permitted operation	Exactly what may happen?	<code>identity_verified</code> → <code>treatment_access_active</code> , nothing else
Enforcement point	Which component outside the model refuses?	The executor, sole holder of the write credential
Accountable owner	Who answers for the outcome?	Healthcare privacy officer
Failure test	Which test proves the refusal?	Wrong-patient, purpose-switch, and self-approval scenarios
Evidence artifact	What proves what happened?	Receipt bound to the exact proposal digest
Failure response	What happens when something fails?	Revoke, reconcile, and route to the manual fallback

3.3 Seven planes and two mediators

Around the contract, the pattern separates seven planes (Fig. 1). The **boundary** plane authenticates ingress and egress, quarantines untrusted input, and exposes a no-read-back seam that certified one-way hardware can replace. The **governed data** plane keeps stable identifiers, replayable events, and versioned snapshots, encrypted at rest under per-subject keys. The **intelligence** plane holds a semantic router, replaceable attested models, agents, and retrieval, but no credential or key. The **authority** plane issues purpose-, time-, and holder-bound grants and approvals, holds model manifests, and provides key custody. The **execution** plane rechecks policy, identity, version, approval, and replay before using the write credential. The **evidence** plane binds intent to outcome. The **resilience** plane fails securely, reconciles uncertain effects, revokes, and routes work to fallback.



Heavy blue outline: a mediator holding what the model lacks. Mediators, contract, keys, and evidence store form the declared trusted base.

Fig. 1. Target deployment architecture, conditional on trusted-service isolation. Intelligence works on vault tokens and can only propose actions and request data. Every request crosses the 7-field control contract to one of two mediators, each the sole holder of what the model lacks: the execution plane holds the write credential (Rule 1); the context gate holds record access and data keys (Rule 2). The semantic router advises which attested model to use; the gate decides where data may go. Logical planes do not prove process isolation or full API integration.

Two components are mediators in the reference-monitor sense. The **executor** enforces the first rule; the **context gate** enforces the second. Together with the contract, signing keys, key custody, evidence store, and identity provider, they form a declared, deployment-dependent trusted base. This is the architecture's central economy: assurance effort concentrates on explicitly inventoried enforcement and infrastructure dependencies, including shared hosts, networks, parsers, CI, administrators and model-serving infrastructure.

Planes are logical responsibilities, not products. The reference stack maps them to FastAPI, PostgreSQL or Redis, Kafka, PySpark, Iceberg, object storage, Ollama-served local models, signing, and a software one-way seam. Adopters may replace any component, but a replacement inherits no assurance until the same conformance and falsification suites pass on it, so the reference stack cannot become a vendor dependency disguised as sovereignty.

3.4 Threat model and trusted computing base

The adversary may control the model, planner, router, retrieved documents, generated code, tool responses, and colluding delegated agents. It may fabricate authorization text, request other subjects' records, replay approvals, poison retrieved evidence, or try to export protected outputs. The claim assumes that authenticated identities, policy distribution, enforcement code, key custody, and the substrate that isolates them are functioning. The model is outside the trusted computing base; its serving process, host, and network isolation can still be trusted dependencies for confidentiality. Python object boundaries in one process do not contain hostile Python execution.

Table 2. Required trust boundaries and their failure consequences. These are deployment obligations, not isolation established by the offline suite.

Trusted dependency	Required separation	Failure consequence and response
Executor and context/release gate	Separate workload identities, least-privilege accounts, authenticated requests; no model-controlled policy input	Compromise can authorize harm. Disable affected automation, revoke credentials, and restore from a reviewed build
Policy, identity and grant issuers	Versioned policy, two-person promotion for privilege expansion, protected signing keys	A validly signed bad policy remains bad. Independent policy review and rollback are required
Host, network, tool adapters and key service	No model credentials, privileged mounts, direct record ports, arbitrary outbound sockets, or administrative APIs	Bypass defeats complete mediation. Demonstrate denial from the actual model container and every worker
Evidence notary and checkpoint witness	Writer cannot replace pinned witness state or signing keys	A recomputed or rolled-back chain can appear valid. Compare externally retained sequence and head hash
Administrators and reviewers	Named powers, separation of duties, time-bounded emergency access, independently reviewed changes	Insider collusion and mistaken approvals remain residual. Contain affected scope and preserve redress

Two mediators mean two enforcement responsibilities, not two mutually independent security domains. Shared policy, hosts, administrators, or credentials create common-mode failures. Splitting services without reducing these dependencies provides little additional assurance. The smallest deployment that satisfies the use case should be preferred: a transactional store and two narrowly permissioned interfaces can be sufficient; Kafka, Spark, Iceberg, a hardware diode, and multiple models are optional integration choices requiring their own justification.

3.5 Invariants and composition rules

For request q at decision time t , define effective authority as the intersection of the authenticated actor's grant, the beneficiary's grant, every ancestor grant, and the current institutional policy. A read, action, or release is permitted only if its subject, tenant, operation, fields, recipient, purpose, zone, expiry, and budget are inside that intersection. Missing dimensions deny; unknown policy versions deny. Purpose strings are exact identifiers unless an institution has explicitly defined and tested a purpose hierarchy. Natural-language claims that one purpose is a harmless subtask do not establish attenuation.

For information flow, an output inherits the restrictions of all context, memory, retrieval, and tool results that could have influenced it. Confidentiality constraints accumulate; integrity endorsement must be checked separately because low-trust content must not become authoritative evidence. Combining allowed-purpose or recipient sets narrows them by intersection, while affected-subject and provenance sets accumulate. An empty permitted set denies. These rules specify a product policy model, not a claim that all dimensions already exist in the current label object. Independent declassification binds the exact output digest, recipient, purpose, approving identity, policy version, and expiry, and is checked again at release.

The two interfaces must compose. A proposal carries trusted references to context receipts and input versions, rather than self-reported provenance. If new context changes the proposal or its authority basis, prior approval is invalidated. Re-identification and outward tool calls are disclosures even when called formatting, logging, or notification. Conversely, an authorized disclosure never grants permission to mutate the subject's record. Enforcing these rules across interfaces is a required end-to-end experiment; separately passing action and disclosure suites does not establish their composition.

4. One governed request in ten steps

A single request passes through ten steps (Fig. 2). They run identically in every domain; only the domain pack changes what each step checks.

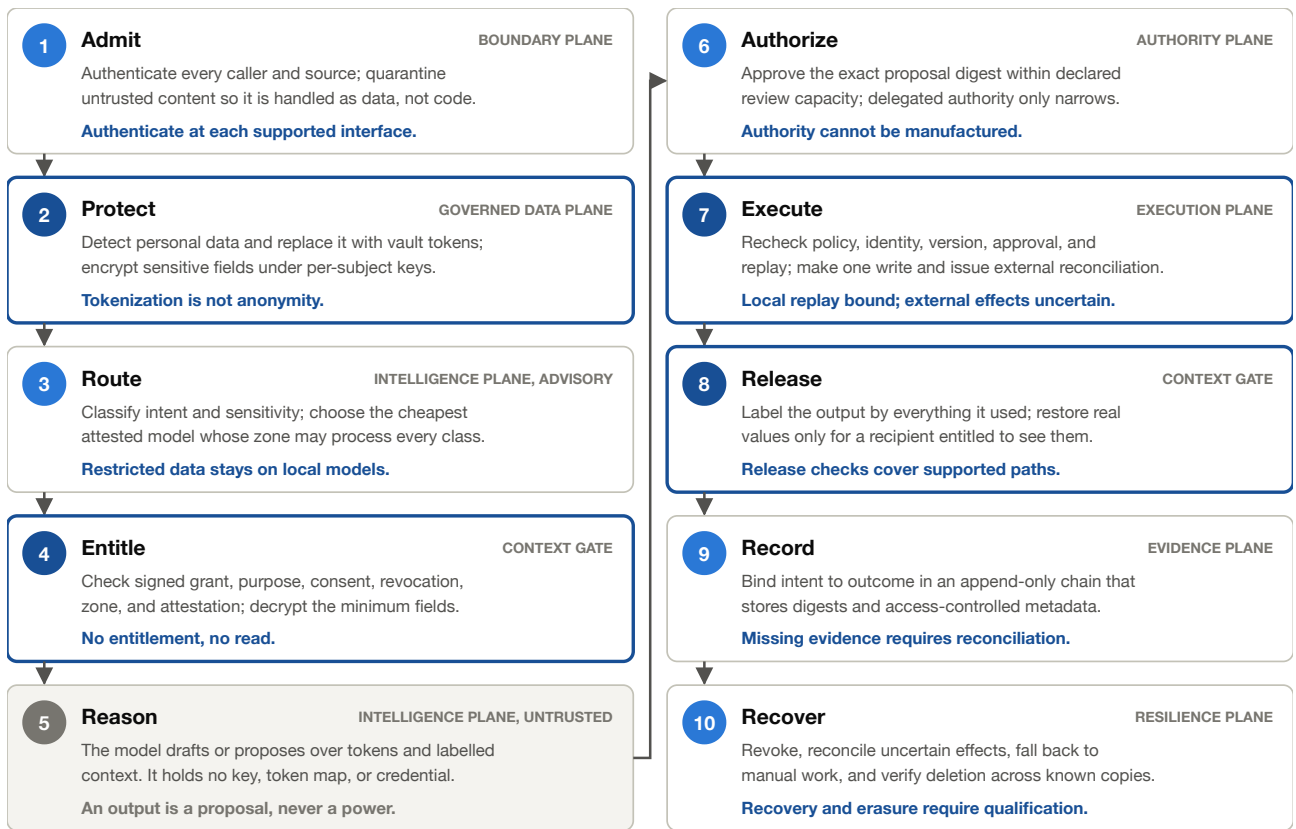


Fig. 2. Target lifecycle in ten steps, requiring end-to-end integration evidence; only the domain pack changes what each step checks. Steps outlined in dark blue require independently enforced boundaries, qualified on the deployed stack. The grey step is the only one performed by untrusted intelligence, and it produces a proposal or a draft, never an effect or a disclosure.

Steps 1–2: admit and protect. The boundary authenticates every caller and source and wraps retrieved content as untrusted data, so an injected instruction is text to be reasoned about, never a command. The optional privacy wrapper replaces declared identifiers and supported detected contact details with vault tokens. Detection is incomplete: free text, rare attributes, images, audio, and prior model knowledge can identify a person without a declared identifier. Tokens are keyed, scoped to a session, and never reproduce the identifier they stand for; the token map lives with the gate. Sensitive fields are stored under envelope encryption: each subject has a data key, wrapped by a key-encryption key for its data class and held by key custody [10], and each ciphertext is bound to its subject and field so it cannot be moved to another record. Tokenization is chosen over redaction because a model needs to reason about *which* patient or applicant a fact concerns without learning *who* they are.

Steps 3–4: route and entitle. A semantic router classifies each request's intent and the sensitivity of the data classes it needs, and ranks approved model candidates by configured cost and capability, subject to processing-zone restrictions; restricted classes default to a local model on institutional hardware. Routing is advisory. The context gate then decides: it verifies a signed grant binding holder, purpose, subjects, fields, classes, and expiry; checks live consent and revocation; confirms that the chosen endpoint's zone may process every class; and checks a registered, signed manifest against the runtime-reported artifact digest when the registry is bound to this path. A production profile must also enforce manifest expiry, purpose, data classes, authenticated endpoint identity, and revocation; a digest supplied by the model itself is not evidence of loaded weights. Only then does the gate read the record source and decrypt the minimum necessary fields. A subverted router cannot expand exposure only if all inference traffic passes this independently configured gate and the runtime and network honor the declared endpoint boundary.

Steps 5–6: reason and authorize. The model drafts or proposes over tokens and labelled context. It holds no key, no token map, and no write credential, so its output is a proposal, never a power. An approval binds to the exact proposal digest covering target, transition, evidence and resource versions, requester, role, audience, and expiry; any change needs new authorization. Approvals can be signed with Ed25519, so the executor holds only a verification

key and cannot mint the approvals it checks. No principal may pass on authority it does not itself hold, so delegation chains only attenuate. Each hop signs its purpose, which may not change below it, and revoking a hop revokes every chain that passes through it at the next use. Human review is declared as capacity with a deliberation floor, escalation, and fallback, so demand beyond capacity defers instead of turning approval into ritual.

Steps 7–8: execute and release. The executor rechecks policy, identity, version, approval, and replay. In the transactional profile, the governed mutation and evidence commit together. A retry returns the existing outcome or enters reconciliation; this is not an exactly-once guarantee for arbitrary remote tools. Every output carries the join of the labels of everything its session received, so a model cannot declassify its own summary. At release the gate rechecks consent, revocation, and expiry, requires the recipient to dominate the output's label, and restores real values in place of tokens only for a recipient entitled to that subject. Lowering a label requires an exact-output declassification by an independent declared role.

Steps 9–10: record and recover. The evidence plane binds intent to outcome through a hash chain. Existing payloads can include identifiers, rationale, or other sensitive metadata; digests alone would also expose low-entropy facts to guessing. The target evidence profile therefore minimizes payloads, encrypts sensitive records, controls inspection, and anchors checkpoints outside the writer's administrative boundary. The reference notary signs the record count and head hash with a key the evidence writer does not hold, so a history that an insider rewrote and re-chained still fails verification. Resilience revokes grants, reconciles uncertain effects, and routes work to a manual fallback. The optional erasure service destroys governed subject keys and probes supplied copies. This is scoped verification, not proof that every backup, exported key, cache, model memory, or recipient copy has been erased. Cryptographic erasure [11] requires a complete copy and key inventory and verified destruction of recovery paths; replaying a deletion journal through one restore API does not destroy a separately retained decryptable key backup.

5. One kernel, many domains

Institutional configuration is supplied by a **domain pack**: a declarative file that supplies purposes, data classes and their processing zones, recipients, transitions and their approvers, declassification rules, emergency access, and fallback. Policy leaders author its meaning; engineers bind it to enforcement. A new sector replaces the pack, not the kernel, and regenerates its own evidence. Table 3 shows the same steps expressed in five sectors, drawn directly from the released packs.

Table 3. The same architecture in five sectors. Declared policies in the released packs; education currently has action fixtures, while the four other columns also have disclosure fixtures. This table does not imply equal end-to-end coverage.

Step	Healthcare	Corporate data	Consumer finance	Public benefits	Education
Declared purposes (4, 8)	treatment, research, emergency treatment, patient access	internal analysis, external release, legal review	credit decision, fraud investigation, customer access, regulatory reporting	eligibility, appeal, applicant access, statistics, urgent safeguarding	student-support casework, academic record correction
Most restricted data and where it may be processed (2–4)	diagnosis, mental-health notes, genomic markers: on-premises model only	contract terms, customer contacts, product roadmap: private model only	identifiers, credit score, transactions, fraud signals: bank-private model only	national ID, disability and immigration status: sovereign model only	governed through action rules and exact approvals
Consequential action and independent approver (6–7)	treatment access: healthcare privacy officer	external release: data protection officer	credit-limit change: credit risk officer	benefit decision: benefit approver; appeal: independent appeals officer	transcript correction: registrar, after instructor confirmation
Governed release (8)	clinician; de-identified to researchers after research review	aggregates to partners after data protection approval	aggregates to the regulator after compliance approval	aggregates to the statistics office after data protection approval	decision returned through the case record

Step	Healthcare	Corporate data	Consumer finance	Public benefits	Education
Exception path (10)	break-glass for emergency treatment, one hour, reviewed	legal hold freezes the dataset	fraud break-glass, thirty minutes, compliance review	urgent safeguarding, one hour, reviewed	appeal reopens a rejected correction
Refused by construction	public model API; external email	public model API; personal email	marketing partner receives nothing	enforcement agency receives nothing	a correction no instructor confirmed

6. Building future AI systems: a seven-stage lifecycle

An architecture becomes a practice only when a team knows what to do on Monday. Figure 3 gives the lifecycle every new AI capability follows, in any sector (Fig. 3). Each stage produces an artifact and passes a gate before the next begins.

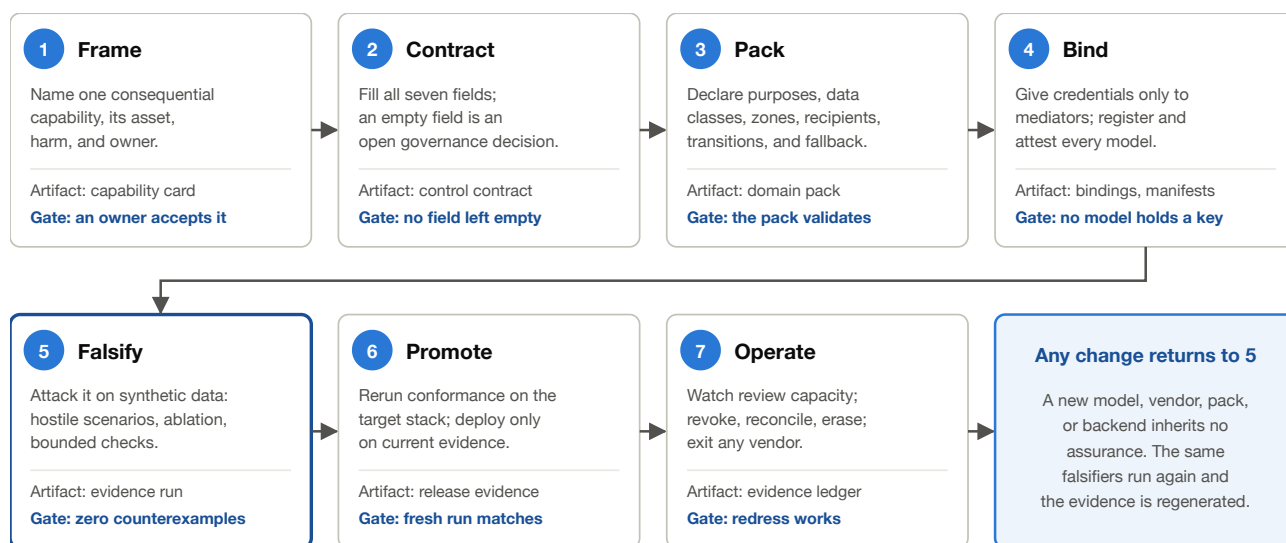


Fig. 3. Building any new AI capability. Each stage produces an artifact and must pass a gate before the next begins. Falsification happens on synthetic data before real records or keys are connected, and any later change to a model, vendor, pack, or backend returns to stage 5, so assurance is regenerated rather than inherited.

- 1. Frame.** Name one consequential capability, the asset it touches, the harm it could cause, and its owner. Start with one capability and a manual fallback.
- 2. Contract.** Fill the seven fields. An empty field is an open governance decision, returned to its owner rather than engineered around.
- 3. Pack.** Declare purposes, data classes and zones, recipients, transitions, declassification, emergency access, and fallback in a domain pack that validates.
- 4. Bind.** Give write credentials, record access, and keys only to mediators; register and sign a manifest for every model endpoint.
- 5. Falsify.** Before real records or keys are connected, run hostile scenarios, ablate every control, enumerate bounded configurations, and run the falsifiers on synthetic data. Resolve all counterexamples to required invariants. For each control, record a matched on/off/restored experiment and its benign utility cost, or state why redundant controls prevent a single-control ablation from identifying its contribution. A redundant defense need not become dispensable to qualify.
- 6. Promote.** Rerun conformance on the target stack and promote only after a fresh run matches committed evidence and the deployment-specific gates in Section 9 pass. Repository tests alone authorize a sandbox demonstration, not consequential institutional use.

7. **Operate.** Watch review capacity, revoke and reconcile, honour erasure, and keep the ability to replace any model or vendor. Any such change returns to stage 5.

The lifecycle gives each audience a concrete role. Policymakers and procurement officers can demand the seven fields, the trusted base, and the regenerated evidence before a contract is signed. Engineers can map each duty to an enforced interface. Auditors can rerun the evidence rather than trust a report.

6.1 GenAI integration contract

A future model, orchestrator, or tool protocol plugs into typed proposal and context-request interfaces. The interface admits bounded data structures, enumerated operations, canonical identifiers, and authenticated caller context. It must reject ambiguous encodings, duplicate security fields, unsupported schema versions, oversized nested values, non-finite numbers, and unresolved destinations. The server derives principal, policy version, classification, and authority; model-generated JSON supplies none of them. Tool descriptions and retrieved instructions remain untrusted even when signed by their publisher: authenticity is not authorization.

Retrieval is a protected read. Enforce tenant, subject, purpose, and field restrictions before retrieving passages, embeddings, reranked candidates, or cached answers; filtering the final prose is too late. Preserve provenance through chunking and ranking. Memory writes, conversation summaries, vector entries, prompt caches, and agent handoffs carry the originating restrictions and retention deadline. A new session cannot wash away restrictions by importing an old summary. Invalidation must traverse derived artifacts when consent, source versions, grants, or retention change. Fine-tuning on governed records is a separate data-use capability; this paper makes no machine-unlearning guarantee.

Images, audio, OCR, attachments, and executable notebooks enter the same boundary. Generated code runs in an isolated environment with bounded resources and no inherited secrets or unrestricted network. Browser, shell, and tool-server adapters require operation-specific policy and canonical destination checks, including redirects and resolved network addresses. Neither a tool allowlist nor schema validation makes arbitrary code safe. A streamed response is a sequence of disclosures: buffer until authorized, or authorize each chunk under a fixed label and recipient context; stop transmission on revocation. Partially transmitted data cannot be recalled.

A model change includes weights, adapters, tokenizer, runtime, system template, decoding configuration, tool catalogue, and retrieval configuration. Record a versioned bundle and rerun security and utility tests when any relevant element changes. Local deployment reduces certain data-transfer dependencies but does not prove confidentiality or correctness. Remote providers require authenticated endpoints, enforceable retention and training restrictions, subprocessor and zone review, and incident and exit procedures. Hardware-rooted attestation can strengthen artifact evidence where available; it cannot establish that an approved model gives good advice.

6.2 Distributed execution and revocation semantics

Each governed operation has a stable tenant-scoped request identity and canonical digest. Within one database, reserve replay identity, compare resource version, authorize, mutate, and append evidence in one transaction. For external effects, persist an outbox intent before dispatch and require a downstream idempotency key or a verifiable operation identifier. After a timeout, query status and reconcile; do not repeat an uncertain irreversible effect blindly. Compensation is a separately authorized action with its own receipt, not an erasure of history. If the remote system supports neither idempotency nor status inspection, automatic retry of that consequential operation remains disabled.

Revocation has a declared linearization point. A transactional authorization decision committed before revocation may complete under the declared policy; one ordered after it must deny. Long-running jobs recheck before irreversible dispatch and each protected release. Across services, publish monotonic revocation epochs, bound cache leases and clock skew, and refuse new consequential use when freshness cannot be established. The deployment must measure the maximum stale-use window rather than promise instantaneous global revocation. Action-delegation and disclosure-grant implementations must be qualified separately; shared vocabulary does not mean shared revocation behavior.

Failures have explicit states: pending, approved, dispatched, committed, uncertain, reconciled, compensated, denied, and expired. Only authorized transitions are legal. Evidence failure before a local transaction commits aborts the mutation; an external effect followed by evidence failure creates an unresolved reconciliation obligation. Admission quotas cover calls, tokens, spend, recursion depth, elapsed time, and per-tenant queue usage. A circuit breaker reserves capacity for recovery and urgent manual service. Break-glass has narrow scope, short expiry, an independent review deadline, and escalation on overdue review; it never creates a general exception to the kernel.

7. Evaluation

7.1 Method

The evaluation asks distinct questions of the reference implementation. Property and bounded checks compare real enforcement against independent predicates over declared configuration spaces; ablation asks whether harm returns when a control is removed; conformance reruns the properties on substituted backends; stateful testing drives random sequences against an independent reference model; comparison arms send identical hostile inputs to weaker designs; and falsifiers try to refute the thesis directly. Utility is always reported beside containment, because a system that denies everything scores perfectly on containment and remains useless. The result generator counts collected tests; collection is not a passing test run. The evidence package records the executed suite outcome separately. Fixture evaluation needs no model weights or external provider calls, although HTTP and process tests can use local services. Section 7 reports synthetic enforcement results, not live-model task quality.

7.2 Mediation against conventional controls

Against an unguarded agent, a safety prompt with a per-agent tool allowlist, and this architecture, the same seven attacks delivered 28, 8, and 0 harmful actions, with every benign task completed in all three arms (Fig. 4a). Against the reference profile, **30 of 30 adversarial scenarios contained, 240 configurations with zero violations, 11 distinct denial controls were reached, 8 of 8 ablated controls restored their harm when removed, and 26 conformance checks passing on 2 backend profiles**; utility held at **6 of 6 benign tasks completed, false-denial rate 0.0**. A 32-caller race produced **1 mutation and 1 receipt**.

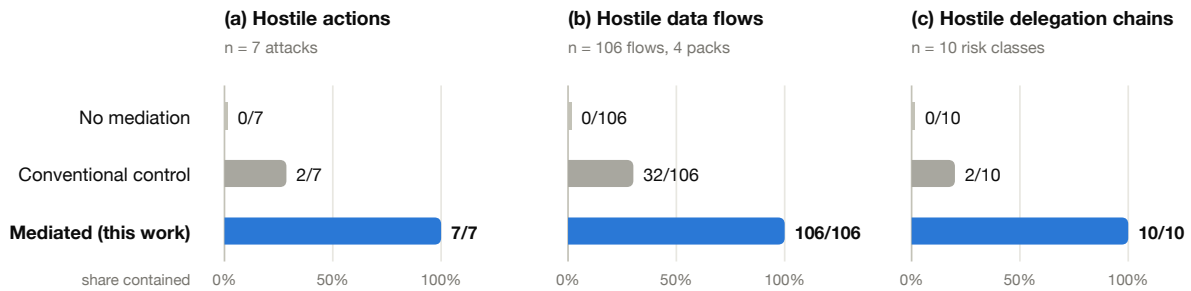


Fig. 4. Containment across three independent suites. Each panel sends identical hostile inputs to no mediation, a conventional control, and this architecture. (a) Actions: the conventional arm is a safety prompt with a per-agent tool allowlist; 28, 8, and 0 harmful actions reached the protected asset, and every arm completed 2 of 2 benign tasks. (b) Data flows: the conventional arm uses signed class-cleared grants and checks the model's claimed output label; 15 of 15 legitimate flows completed under the context gate. (c) Delegation: the conventional arm validates each hop against its immediate caller; a benign two-hop chain completes in every arm.

Reads were tested the same way. Across corporate, healthcare, financial, and government packs, **106 of 106 hostile data flows were contained**, including wrong-patient injections, purpose switches, withdrawn consent, and self-labelled summaries. Conventional access control, with signed class-cleared grants and an output check on the model's claimed label, **contained 32** (Fig. 4b). **All 14 checks were load-bearing, 24,272 read and release configurations showed zero violations**, and 15 of 15 legitimate flows completed. Stateful testing, **800 sequences and 32,000 operations** against an independent reference model, found a defect single-step enumeration missed: a summary drafted before consent was withdrawn could still be released. Release now rechecks consent, revocation, and expiry, and the sequences agree.

Composition was tested with delegation chains. Nine invariants, from attenuation and rootedness to holder binding, are all load-bearing across **768 enumerated chain shapes and zero violations**. Validating each hop against its immediate caller, a plausible design, **contains 2 of 10 risk classes where verifying the chain contains 10** (Fig. 4c). Local correctness at every service is not chain correctness.

7.3 Review capacity

In a simulation using declared reviewer behavior, **a roster of 11 reviewers sustains 2,640 consequential actions per day**. In a queue trial where 40 arrivals reach one reviewer, **Without load control, 4 such merit failures execute. With it, 0 do, and 32 actions defer to manual review** (Fig. 5). Across 25 parameter combinations the control was load-bearing in 16 of the 20 where harm was possible, harm reached zero in 16, and repairing contradictory defaults **drove the false-positive cost to 0**. Automation bias makes a hurried reviewer ratify what an assistant recommends [9]. In the declared simulation, reviewer assistance raises throughput and independence reduces correlated error: the simulation produced **5 merit failures with a dependent assistant, 1 with an independent one**, so the architecture refuses to start a dependent assistant at a lowered deliberation floor.

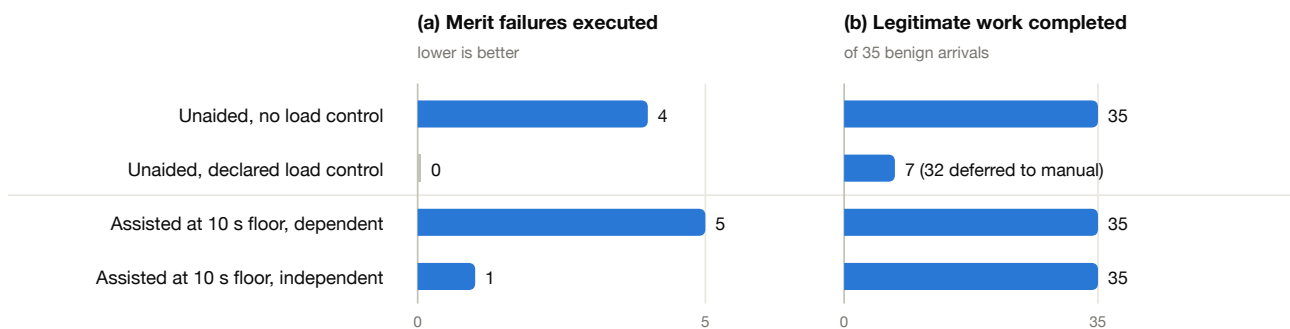


Fig. 5. Human review as a bounded resource. In the queue trial, 40 arrivals reach one reviewer with a declared attentive capacity of 8; 5 proposals are valid in form but wrong on merit. Declared load control stops 4 merit failures from executing at the cost of 32 deferrals. Adding a review assistant and lowering the floor to 10 s restores throughput; a dependent assistant lets 5 failures through and an independent one 1, so the dependent configuration is refused before it starts. Reviewer behaviour is modelled by declared, adjustable parameters.

7.4 Transfer across sectors

One successful domain cannot establish transfer (Fig. 6). **6 independently reported domain packs cover 55,440 bounded configurations, 180 of 180 hostile scenarios contained, 58 of 58 benign tasks completed, and zero unauthorized mutations**. The academic-record pack independently reports **4,800 configurations with zero violations, 30 of 30 scenarios contained, 9 of 9 benign tasks, 26 conformance checks**; its first run exposed a role declared in the pack but ignored at runtime, a defect invisible with a single domain.

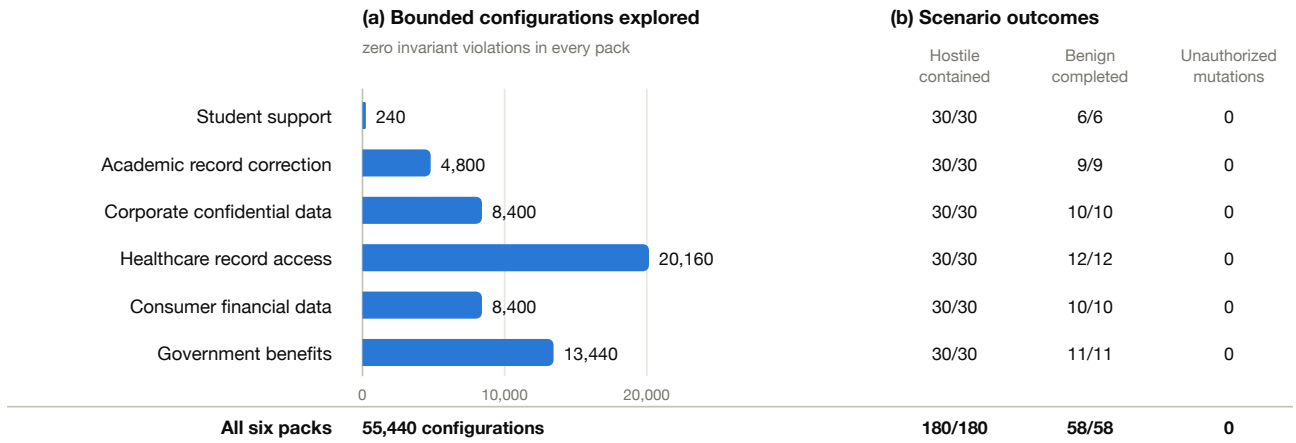


Fig. 6. One kernel, 6 domain packs. Each pack is verified independently against its own denominators: bounded configurations explored with zero invariant violations (a), and hostile scenarios contained, benign tasks completed, and unauthorized mutations (b). Across all packs: 55,440 configurations, 180 of 180 hostile scenarios contained, and 58 of 58 benign tasks completed. A new sector replaces the pack and regenerates its own evidence; the kernel is unchanged.

7.5 A safety case that does not rest on alignment

A catalogue of 36 failure classes, including specification gaming, deceptive risk reporting, oversight flooding, collusion, authority laundering, injection, poisoning, and compromised signers, records **21 contained, 10 bounded, and 5 residual**, each bound to evidence checked to exist (Fig. 7a). Six falsifiers then try to refute the thesis across every pack. Their first run did: five contract bindings named functions that did not exist. Repaired, **104,997 bounded attempts found zero counterexamples**, and removing a single mediator check produces them (Fig. 7b). These fixtures assume hostile proposals and test the corresponding checks. They do not establish containment of every output an arbitrary model might produce, all protocol exploits, or attacks against the trusted runtime.

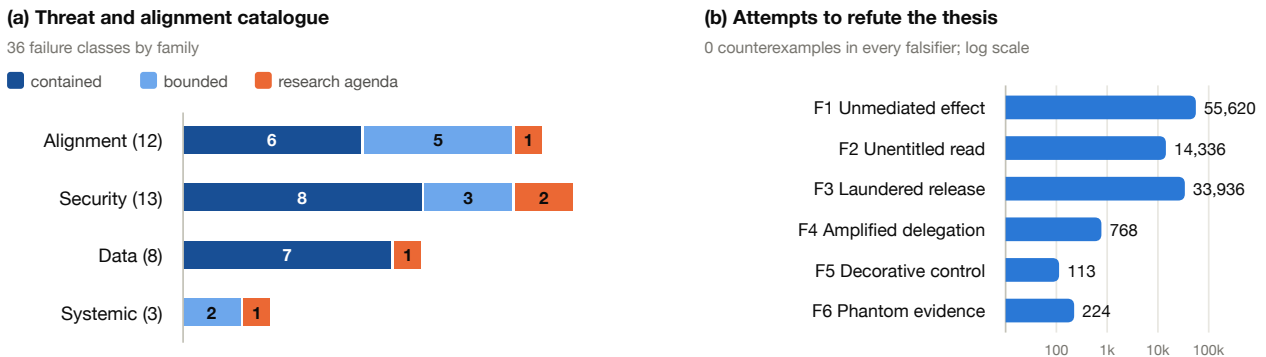


Fig. 7. The safety case does not rest on alignment. (a) Of 36 failure classes, including 12 alignment failures, 21 are contained, 10 bounded, and 5 assigned to the research agenda; every evidence locator is checked to exist. (b) Six falsifiers made 104,997 bounded attempts to produce a governed effect or disclosure from model output alone and found none; removing a single mediator check produces counterexamples.

The method also applies to itself. Applying the contract to the repository's own requirements found that **eighteen of twenty-eight were prose bound to nothing**. Coverage is now published as machine-verified, organizationally attested, or unverified, **reading 41, 3, 0 in the baseline contract register**, and a named test that does not exist fails the build. These checks found defects missed by earlier tests. Existence of a named test is weaker evidence than execution, meaningful assertions, and coverage of the actual deployment path; the optional privacy path has additional HTTP integration tests with separate scope and denominators.

7.6 What the evidence can and cannot establish

The unit of analysis varies: scenario, bounded configuration, operation sequence, control ablation, or task. Their denominators must not be added into one coverage percentage. Enumerated configurations are correlated choices in author-defined spaces, not independent samples of future attacks. Zero counterexamples therefore supports only the enumerated invariants and assumptions; it supplies no empirical probability of safety. The contract register measures its declared requirements, not the fraction of the entire architecture deployed correctly. No defensible overall implementation percentage is established by these figures.

A load-bearing experiment fixes input, seed, backend, and expected forbidden effect; runs the control on, off, and restored; inspects actual target state or released bytes; and includes a legitimate task. A denial log alone is insufficient if a side effect already occurred. For redundant controls, use documented combination ablations in an isolated synthetic environment. Ablation switches must be impossible to enable in a consequential deployment profile. The present fixture comparisons should be supplemented with independently authored attacks, protocol fuzzing, malformed inputs, cross-tenant tests, and a named live-model evaluation. Simulation and local fixtures remain labelled as such.

The next evaluation preregisters task selection, attack budget, seed and version lists, exclusion rules, and endpoints before seeing results. Report unauthorized effects, unauthorized bytes released, attack success by class, legitimate completion, false denials, deferral time, recovery time, and unresolved outcomes. Measure authorization, context assembly, execution, encryption, and evidence overhead separately from inference; report sample counts, hardware, concurrency, warm/cold conditions, and latency quantiles. Cost and energy require measurements, not estimates inferred from token counts alone. Independent reviewers should reproduce the artifact and publish failures as well as successes.

7.7 Earlier component demonstration in education

The earlier conference package (conference/) joins action and disclosure control in one governed-learning pack covering student support, transcript correction, financial aid, faculty feedback, and student challenge. These historical fixture results do not validate caller authentication or independent instructor confirmation at the orchestration entry point. Every consequential capability carries a seven-field contract whose failure test must exist before the pack loads. Each mediator is the real component: envelope encryption under per-student keys, session-scoped tokens, a strict model registry, Ed25519 approvals, seven-axis delegation, a bounded review queue, and a signed-checkpoint notary. The model is scripted to be malicious. **25 of 25 falsifiers held** and **34 of 34 attack attempts were blocked**, judged by inspecting model contexts, released bytes, and register state rather than denial codes. **25 of 29 ablations** restored harm when one control, or one declared pair, was removed, and removed it again when restored. The other four are single controls that another control independently backs: the router attack needs residency and model attestation removed together, and replay needs digest binding and single-use approval removed together.

Stateful testing ran **3,000 steps with 0 violations and 0 false denials**, and removing one control produced between 1 and 154 violations. Seeded grammar attacks gave **300 red-team attempts produced 0 successes**; with the context gate removed, 79 of 150 succeeded. The kernel floor rejected a malicious domain pack with **38 findings**. **6 of 6 delegation escalations** and **5 of 5 model-substitution attacks** were refused, the model saw **0 identity values**, after erasure **0 of 9 checked locations** were readable, 128 concurrent callers produced **1 mutation**, and **31 of 31 register claims** passed. An honest and a malicious model met the same boundary: one completed the correction with evidence, and the other was denied with no state change. These are observations of one synthetic pack in one process, regenerated by `python scripts/conference_evidence.py`, and are not included in the six-pack or four-pack denominators above.

7.8 Joined workflow: defects, repairs and bounded adaptive evaluation

An audit of commit 5c587c361cf114125e39344caa6802efd4eaff11 plus preserved working changes executed 841 baseline tests successfully. The documented reviewer command nevertheless failed its supplemental resilience artifact comparison. Before repair, the education driver accepted a request for an unassigned student, changed that student's grade and released the identified result. It also accepted a caller-selected grade A without independently confirmed evidence. These observations concern an implementation gap in Sections 3.5 and 8.1, not a forged signature or evidence that every existing policy check failed. The legacy driver now refuses these cases and explicitly identifies its simulated identity and reviewer decisions.

The new `joined_workflow.py` provides a deliberately narrow, durable vertical slice. Bearer authentication establishes the actor; an independently seeded grant selects tenant, subject, beneficiary, purpose and operations. A read issues a server-stored context reference with record, source, policy and recipient versions. A proposal retains those bindings and an enumerated value; arbitrary rationale is discarded. A separately authenticated instructor confirms the authoritative source, and a registrar approves that exact proposal and confirmation. Execution rechecks current context and rooted authority within a SQLite write transaction. A separately authenticated recipient receives only a server-rendered result after a fresh release check. Effects, cumulative ancestor budgets and evidence commit atomically. Unsupported tools, network requests, arbitrary memory writes, streams, schema overrides and generated code are denied at this JSON interface. A separately executed same-OS-user subprocess directly read the synthetic database. This confirms that the teaching host does not isolate generated code; the JSON boundary must not be represented as an operating-system security boundary.

Local effects use stable proposal identities and durable receipts; a lost response is reconciled after reopening the database. The demonstration simulates the lost response after an actual SQLite commit; it does not simulate a remote service as evidence of remote exactly-once execution. Revocation is ordered by SQLite transaction acquisition. After revocation commits, a new release denies. Previously released bytes remain disclosed. An appeal officer can issue a separately recorded correction, invalidating the old result's release version. A legitimately authorized bad source still produces a wrong result: this is a substantive error, not an unauthorized mutation. The appeal experiment establishes recoverability of this local record, not recovery from every consequence of the original decision.

The experiment compares static sequences, random requests, feedback-driven search, and a genuinely trained tabular Q policy. The latter trains for 1,200 episodes using sparse reward from independently queried backend outcomes, with an eight-request and five-second cap per episode, learning rate 0.4, discount 0.9 and exploratory probability 0.25; every fifth training episode uses a scripted exploration trajectory. Training removes the source-confirmation check. The frozen policy is evaluated on 40 new seeds per arm and strategy. Seed sets differ, but grammar and task families overlap: these are not unseen exploit classes. The attacker selects enumerated requests and cannot edit policy, source data, defender or oracle through that interface. The evaluator holds synthetic reviewer credentials; their decisions are simulations, not a human or review-LLM study.

Table 5. Unsupported corrections in 40 episodes per cell. The server-authorization baseline retains authentication, scope checks, reviewer roles, exact-proposal binding, replay, release and budgets, but does not check source correctness. Thus its failures below are approved substantive errors; none is counted as unauthorized execution. The comparisons are local implementations, not evaluations of CaMeL, FIDES or commercial systems.

Strategy	Joined workflow	Server authorization	Deny all
Static	0/40	40/40	0/40
Random	0/40	0/40	0/40
Adaptive search	0/40	40/40	0/40
Trained tabular Q	0/40	40/40	0/40

Each arm/strategy also runs one legitimate workflow: joined and server authorization complete 1/1, deny-all completes 0/1. Unauthorized subject mutations and unauthorized sink bytes are zero in each bounded run. The limited random budget failing to discover the removed control is a sensitivity failure, not evidence that the control is

unnecessary. The on/off/restored regression produces grades C/A/C for the unsupported request, while legitimate work still completes. Raw trajectories, seeds, Q values, timings and manifests are in `audit/adaptive-results.json`; unlike denominators are not pooled. Language-model reinforcement learning, an independently authored held-out attack corpus, and trained reasoning-monitor evaluation are NOT RUN. Separately, four fresh local `llama3.2:1b` prompts ran against the fixed proposal interface: final record values were C, C, C and B, with no release. The legitimate prompt produced truncated invalid JSON and failed to complete; hostile prompts yielded a missing value, an unsupported A refused at confirmation, and a supported B respectively. This small local smoke test is not frontier-model robustness evidence. Raw outputs are retained in `audit/live-local.json`.

7.9 Observable reasoning and integration limits

Five synthetic rationale variants include absent reasoning, reassuring claims, fabricated institutional authority, false citations and misleading summaries. All accompany the same unsupported proposal and fail independent confirmation. No private hidden chain-of-thought is requested. Rationale is neither retained in evidence nor used as authority; there is no optional monitor to fail open. This demonstrates independence from those rationale strings, not immunity to persuasive models. A colluding or mistaken trusted source and reviewers can still authorize a harmful decision, as the appeal regression shows.

The trusted computing base includes the JSON parser, canonicalizer, Python and SQLite runtimes, token configuration, source and policy administrators, reviewer authentication, evidence writer, operating system, filesystem and CI. Keys and evidence share the teaching host; a co-located manifest detects accidental edits but cannot defeat an attacker replacing both manifest and result. A retained checkpoint detects truncation relative to that checkpoint; trusted-host rollback of the checkpoint itself remains outside the tested guarantee. The joined store is synthetic and unencrypted and is not an implementation of the separate privacy-vault profile. Deploying the broader HTTP, model, Kafka, Spark, Iceberg, cache and tool paths requires separate qualification; the new slice does not silently inherit their coverage.

8. Education as the dissemination mechanism

The conference theme matters because this architecture requires **system literacy**: distinguishing proposals from powers and requests from entitlements, locating the mediator, following a token back to the entitlement that permits it, and reconstructing an outcome and its redress. A two-hour offline lab has participants break a boundary, remove a control, watch the harm return, and transfer the contract to their own domain.

For *AI for Learning*, the education packs govern institutional workflows such as student support and transcript correction with the same rules as a hospital or a benefits agency. For *Learning for AI*, the contribution is a reusable method for learning how AI systems acquire and exercise power. The repository is offered as an openly licensed public-interest resource consistent with the aims of the Global Digital Compact [8], without claiming formal digital-public-good recognition: any institution can inspect the rules, replace the pack, and publish its own evidence, and educators can teach the same boundaries to policymakers, engineers, and students [7].

8.1 Education workflow and study protocol

The education extension should join two current strengths: action control for transcript correction and disclosure control demonstrated in other sectors. A student-support request first releases only the assigned student's necessary academic fields; counselling, disability, and financial-aid information require separately justified access. The model drafts an intervention with source references and uncertainty. It cannot change a grade, decide aid, or diagnose a student from that read grant. Transcript correction requires the instructor's confirmation and the registrar's exact-proposal approval against the current record version. A changed student, grade, source, or policy requires a new approval. The earlier governed-learning driver did not independently confirm instructor facts or enforce an assigned-student boundary at admission. The repaired joined SQLite workflow in Section 7.8 implements these checks with separately authenticated synthetic roles; they are not included in the four-pack disclosure denominator.

A learner inspects the request, principal, model/version, permitted fields, proposal, grant, mediator decision, resulting state, evidence, and appeal route. They predict the outcome of an attack before running it, inspect actual state after denial, disable one control on synthetic records, restore it, and transfer the contract to an unfamiliar domain. The exercise must show both a successful legitimate operation and the cost of a false denial. Attack-lab credentials and fixtures cannot access institutional systems; the normal deployment has no interactive disable-control feature.

Educational effectiveness requires a study. Compare the lab with a matched explanation-only condition; use consented participation, baseline knowledge assessment, blinded scoring of boundary identification and a novel transfer task, delayed retention assessment, and accessibility accommodations. Report cohort sizes, attrition, uncertainty, and scoring agreement. Do not infer learning from clicks or self-reported confidence. Operational student outcomes require separate evaluation: correction accuracy, subgroup error and denial rates where lawful, time to support, appeal timeliness, and correction quality. An affected student receives an accessible decision explanation and independent appeal route, without exposure of another student's evidence or confidential security internals.

9. Qualification, unresolved hurdles, and acceptance gates

This revision distinguishes a design resolution from evidence that the resolution works. Sections 3 and 6 specify the missing mechanisms; Table 4 states what must be shown before relying on them. A written requirement closes an ambiguity in the framework, not a deployment risk. No experiment in this paper proves immunity to compromised administrators, a compromised mediator, inference from legitimately disclosed attributes, or substantively wrong institutional policy.

Table 4. Gap closure obligations. Each row requires a named institutional owner and versioned evidence before promotion.

Hurdle	Required mechanism and accountable role	Closure evidence and stop condition
Bypass and common-mode compromise	Security owner isolates mediator identities, ports, storage, keys, workers, and control plane	Negative tests from actual model and worker containers; any direct governed read/write blocks promotion
Policy and domain-pack escalation	Policy owner signs versioned packs; schema validation plus immutable kernel floor; peer review of privilege expansion	Malicious-pack and rollback tests against instantiated runtime; unknown or stale policy denies
Tenant, beneficiary and delegation scope	Identity owner binds authenticated actor and beneficiary; intersects rooted grants and tenant/resource scope	Cross-tenant, sibling-agent, purpose, expiry and ancestor-revocation tests; no inherited administrator privilege
Label and action composition	Data owner binds proposal to context receipts and propagates restrictions across every interface	End-to-end wrong-subject, stale-context, changed-proposal and release tests; missing provenance denies
Memory, retrieval and modality coverage	Data owner inventories caches, vectors, summaries, streams, OCR and handoffs; labels and invalidation follow derivation	Seeded sensitive-record probes across all paths, including cache hits; unknown lineage quarantines
Declassification and inference risk	Privacy owner approves exact outputs; reviews quasi-identifiers, small cohorts and repeated-query inference	Re-identification and differencing tests; high-risk exports held for review; tokens are not anonymity
Erasure and evidence retention	Records owner inventories keys and copies, propagates deletion and legal-hold decisions, protects deletion journal	Restore from every supported backup and attempt decryption independently; unread or unlisted copies remain unresolved
Model and tool supply chain	Platform owner pins approved bundles, endpoint identities, dependencies and credentials	Substitution, lying-runtime, redirect and malicious-tool tests; runtime claims alone cannot qualify hardware attestation
Distributed and external effects	Service owner uses transactional state/evidence, outbox, idempotency and reconciliation	Revoke/dispatch races, crash-at-each-boundary, duplicate delivery, stale version and backup-restore exercises

Hurdle	Required mechanism and accountable role	Closure evidence and stop condition
Evidence integrity and privacy	Assurance owner retains externally witnessed checkpoints; restricts and encrypts evidence access	Rewrite, truncation, rollback, key-rotation and unauthorized-inspection tests; no witness means no rollback claim
Review overload and correlated errors	Service owner declares measured capacity, queue deadlines, escalation and manual staffing	Workload and reviewer study, dependent-assistant tests and overdue escalation; overload never auto-approves
Correctness, fairness and redress	Domain owner validates policy and source quality with affected users; independent appeal officer	Blinded merit assessment, accessibility and subgroup analysis, completed appeal exercises; unresolved harmful policy blocks use
Availability, cost and sustainability	Operations owner measures budgets, fallback capacity, recovery objectives, cost and energy	Sustained-load and outage drills at declared targets; unacceptable service delay stops expansion
Transfer and external assurance	Adopting institution qualifies exact model, backend, domain, reviewer regime and administrative boundaries	Independent review, signed limitations and go/no-go record; changing a critical dependency reopens relevant gates

9.1 Evidence levels and implementation status

The action, disclosure, delegation, replay, and contract-binding suites provide runnable reference evidence. Human degradation and review-assistant independence experiments are simulations with declared parameters. The opt-in in-memory HTTP privacy profile binds the wrapper, vault, encrypted record source, and strict model registry to context, model-task, output, and release routes. Integration tests exercise authorized token restoration, wrong-holder and recipient denial, model substitution, expiry, revocation, prompt/output tokenization, and denial of inference without governed context. The default profile remains disclosure-only and reports that status. The privacy profile refuses a durable disclosure store because its vault and key custody are not yet persistent. The notary and stricter pack floor are bound in the education demonstration world and its evidence generator, not yet in the HTTP deployment path; their tests must not be presented as enforcement on every route.

The audit also identified why numeric alignment alone was insufficient: the full-paper renderer described its prose as guarded, but the existing alignment suite primarily checked the abstracts and deck. A full-paper check must bind its quoted metrics to regenerated results, while the claims register records whether each additional assertion is a fixture result, source-only component, deployment requirement, or open empirical question. Passing a substring check is not semantic review. The attached revision report records executed checks separately from collected test counts and retains unresolved integration gaps.

9.2 Second-audit implementation findings

The second audit found defects that the first regression suite did not exercise. The vault retained identity values as forward-lookup keys, sequential replacements could corrupt emitted tokens, and cached token lookup did not recheck erasure. Forward lookup now uses a keyed canonical digest, replacements preserve existing tokens, allocation is bounded, and erased-subject entries are purged. This removes live lookup retention; it does not promise zeroization of Python memory or deletion of copies held elsewhere.

Record loading now stages all fields before committing changes, and snapshot restore preserves the version high-water mark. Declassification preserves the exact source-session identifier instead of splitting it at a slash, applies case-insensitive literal redaction, and removes identity tokens when the rule removes identity. Redaction still does not prove anonymity. Strict registry use requires purpose and classes; re-registering a revoked manifest requires explicit administrative reinstatement. Release evidence binds the digest of the bytes actually returned after identity restoration. Model errors no longer echo arbitrary exception payloads to HTTP callers.

Erasure verification now probes restored keys in an isolated custody instance rather than replacing live keys. It reports bad probe credentials and negative literal scans as unverified, and custody rejects journal rollback relative to known history or the backup's recorded journal length. A fresh restore still needs an independently retained current

journal or watermark to establish completeness beyond that bound. Finally, failed test collection stops evidence generation instead of publishing a partial count. These repairs are covered by the separate privacy-integration suite; production isolation, unlearning, institutional outcomes, and independent assurance remain unverified.

A further pass closed two defects the earlier suites did not exercise. First, the demonstration's approval signing key was reconstructible: the asymmetric authority derived its private key from the *public* key identifier, and the education world seeded it from a constant in source, so a party who read the open repository could mint an approval that executed a wrong-student grade change — Rule 1 resting on a key that was not in fact secret. The key is now generated per instance and the executor holds only the public verification key; forging from either former seed is refused, and the assumption that the private key lives in a signing service the model cannot reach is now stated rather than silently contradicted. Second, on the earlier education execution path an approved action still executed after the grant behind its context read had been revoked; that path now re-runs the gate's currency, consent, and scope checks over the grant before the write and refuses with a stable code, matching the composition the joined workflow enforces on its own path. An additional offline adaptive attacker (fssaira conference adaptive), scored by an independent effect oracle rather than denial logs and paired with a positive control that removes a mediator to prove it is not a no-op, found 0 forbidden outcomes with every control on; a learning-based and a frontier-model attacker remain not run, consistent with Section 7.8 [20, 21, 22, 32, 33].

9.3 Promotion and operational governance

Use three explicit stages. A teaching sandbox contains synthetic data and no institutional credentials. A shadow pilot may draft and compare outcomes under approved data access but performs no autonomous consequential mutation. A bounded operational pilot enables only named capabilities after Table 4's applicable evidence is accepted. The institution signs thresholds for utility, unacceptable harm, maximum stale-authorization interval, review backlog, recovery objectives, and appeal turnaround before the pilot begins. Thresholds are domain decisions; this paper does not invent universally safe values.

Promotion requires an inventory of every effect and disclosure path, exact code/configuration versions, a dependency lock or environment export, protected keys and identities, successful negative and legitimate tests, incident and recovery exercises, accessible redress, and independent assessment proportionate to consequence. Record the owner, evidence digest, date, expiry and accepted residual risk for each gate. A failed or missing mandatory gate keeps the capability in the preceding stage. During operation, policy drift, a harmful unexplained outcome, missing evidence, overdue emergency review, or breached capacity limits triggers a scoped pause and manual service. Expansion is a new assurance decision, not a default consequence of technical availability.

10. Conclusion: directions for future AI systems

1. **Build on the Mediation Thesis.** Treat intelligence as untrusted, mediate every effect and data flow, and publish the attempts to refute it.
2. **Govern reads as strictly as writes.** Bind every context to purpose, subject, field, consent, and location, and label every output by what produced it.
3. **Minimize identification risk.** Tokenize declared identifiers, hold keys outside intelligence, test inference and indirect leakage, and restore identity only for an entitled recipient.
4. **Make every consequential capability contractable.** No capability ships without the seven fields, a test bound to something that runs, a recovery owner, and a conformance claim.
5. **Treat sovereignty as a demonstrable lifecycle capability.** Measure control of keys, data movement, model replacement, evidence, recovery, and exit, not hosting location.
6. **Publish authority and oversight ceilings before automation roadmaps.** Delegation depth and review capacity are safety limits, not staffing footnotes.
7. **Build sectors as domain packs over a shared kernel,** and trade failure cases, not sensitive data.

An AI system should be governed less like an application and more like an institution: with a constitution, separated powers, a memory it cannot rewrite, and a safe way to stop. The reference implementation supplies bounded evidence for this direction; deployment qualification and independent field evaluation remain necessary. The joined audit demonstrates why a passing fixture suite must be challenged at the orchestration boundary. Exact authorization is necessary but does not establish factual correctness; independently confirmed provenance and accessible recovery narrow that gap without eliminating trusted-source error. A useful foundation makes those obligations executable and visible rather than declaring them solved.

References

- [1] Rose, S., Borchert, O., Mitchell, S., and Connelly, S. (2020). *Zero Trust Architecture*. NIST SP 800-207. <https://doi.org/10.6028/NIST.SP.800-207>
- [2] Autio, C., et al. (2024). *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*. NIST AI 600-1. <https://doi.org/10.6028/NIST.AI.600-1>
- [3] OWASP GenAI Security Project (2025). *OWASP Top 10 for Agentic Applications 2026*. <https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/>
- [4] OWASP GenAI Security Project (2026). *Agent Control Standard*. <https://genai.owasp.org/resource/agent-control-standard-acs/>
- [5] MITRE (2024). *ATLAS: Adversarial Threat Landscape for Artificial-Intelligence Systems*. <https://atlas.mitre.org/>
- [6] ISO/IEC (2023). *ISO/IEC 42001:2023 — Information technology, Artificial intelligence, Management system*. <https://www.iso.org/standard/81230.html>
- [7] Miao, F., and Cukurova, M. (2024). *AI competency framework for teachers*. UNESCO. <https://unesdoc.unesco.org/ark:/48223/pf0000391104>
- [8] United Nations (2024). *Global Digital Compact*. <https://www.un.org/pact-for-the-future/en/annex-i-global-digital-compact>
- [9] Parasuraman, R., and Manzey, D. (2010). Complacency and bias in human use of automation: an attentional integration. *Human Factors*, 52(3), 381–410. <https://doi.org/10.1177/0018720810376055>
- [10] Barker, E. (2020). *Recommendation for Key Management: Part 1 — General*. NIST SP 800-57 Part 1 Rev. 5. <https://doi.org/10.6028/NIST.SP.800-57pt1r5>
- [11] Chandramouli, R., and Hibbard, E. (2025). *Guidelines for Media Sanitization*. NIST SP 800-88 Rev. 2. <https://doi.org/10.6028/NIST.SP.800-88r2>
- [12] Saltzer, J. H., and Schroeder, M. D. (1975). The protection of information in computer systems. *Proceedings of the IEEE*, 63(9), 1278–1308. <https://doi.org/10.1109/PROC.1975.9939>
- [13] Denning, D. E. (1976). A lattice model of secure information flow. *Communications of the ACM*, 19(5), 236–243. <https://doi.org/10.1145/360051.360056>
- [14] Clark, D. D., and Wilson, D. R. (1987). A comparison of commercial and military computer security policies. *IEEE Symposium on Security and Privacy*, 184–194.
- [15] DeBenedetti, E., et al. (2025). *Defeating Prompt Injections by Design*. <https://arxiv.org/abs/2503.18813>
- [16] Costa, M., et al. (2025). *Securing AI Agents with Information-Flow Control*. <https://arxiv.org/abs/2505.23643>
- [17] Greenblatt, R., Shlegeris, B., Sachan, K., and Roger, F. (2024). AI control: Improving safety despite intentional subversion. *Proceedings of the 41st International Conference on Machine Learning*. <https://arxiv.org/abs/2312.06942>
- [18] European Parliament and Council (2016). *Regulation (EU) 2016/679 (General Data Protection Regulation)*. <https://eur-lex.europa.eu/eli/reg/2016/679/oj>
- [19] European Parliament and Council (2024). *Regulation (EU) 2024/1689 (Artificial Intelligence Act)*. <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>
- [20] Perez, E., Huang, S., Song, F., Cai, T., Ring, R., Aslanides, J., Glaese, A., McAleese, N., and Irving, G. (2022). Red teaming language models with language models. *EMNLP 2022*, 3419–3448. <https://aclanthology.org/2022.emnlp-main.225/>
- [21] Nasr, M., Carlini, N., Sitawarin, C., et al. (2025). *The attacker moves second: stronger adaptive attacks bypass defenses against LLM jailbreaks and prompt injections*. <https://arxiv.org/abs/2510.09023>
- [22] Zhan, Q., Fang, R., Panchal, H. S., and Kang, D. (2025). Adaptive attacks break defenses against indirect prompt injection attacks on LLM agents. *Findings of NAACL 2025*, 7101–7117. <https://arxiv.org/abs/2503.00061>
- [23] DeBenedetti, E., Zhang, J., Balunović, M., Beurer-Kellner, L., Fischer, M., and Tramèr, F. (2024). AgentDojo: a dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. *NeurIPS 2024 Datasets and Benchmarks Track*. <https://doi.org/10.52202/079017-2636>
- [24] Shi, T., He, J., Wang, Z., Li, H., Wu, L., Guo, W., and Song, D. (2025). *Progent: securing AI agents with privilege control*. <https://arxiv.org/abs/2504.11703>
- [25] Zhong, P. Y., Chen, S., Wang, R., McCall, M., Titzer, B. L., Miller, H., and Gibbons, P. B. (2025). *RTBAS: defending LLM agents against prompt injection and privacy leakage*. <https://arxiv.org/abs/2502.08966>

- [26] Anderson, J. P. (1972). *Computer security technology planning study*. ESD-TR-73-51, Vol. I. <https://apps.dtic.mil/sti/tr/pdf/AD0758206.pdf>
- [27] Lampson, B. W. (1973). A note on the confinement problem. *Communications of the ACM*, 16(10), 613–615. <https://doi.org/10.1145/362375.362389>
- [28] Sweeney, L. (2002). k-anonymity: a model for protecting privacy. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 10(5), 557–570. <https://doi.org/10.1142/S0218488502001648>
- [29] Narayanan, A., and Shmatikov, V. (2008). Robust de-anonymization of large sparse datasets. *IEEE Symposium on Security and Privacy*, 111–125. <https://doi.org/10.1109/SP.2008.33>
- [30] Dwork, C. (2006). Differential privacy. *ICALP 2006*, LNCS 3927, 1–12. https://doi.org/10.1007/11787006_1
- [31] Helland, P. (2012). Idempotence is not a medical condition. *Communications of the ACM*, 55(5), 56–65. <https://doi.org/10.1145/2160718.2160734>
- [32] Baker, B., Huizinga, J., Gao, L., Dou, Z., Guan, M. Y., Madry, A., Zaremba, W., Pachocki, J., and Farhi, D. (2025). *Monitoring reasoning models for misbehavior and the risks of promoting obfuscation*. <https://arxiv.org/abs/2503.11926>
- [33] Korbak, T., Balesni, M., Barnes, E., Bengio, Y., et al. (2025). *Chain of thought monitorability: a new and fragile opportunity for AI safety*. <https://arxiv.org/abs/2507.11473>
- [34] Hardy, N. (1988). The confused deputy (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review*, 22(4), 36–38. <https://doi.org/10.1145/54289.871709>